

Desafio Prático: O Servidor Seguro (Mecanismos de Defesa)

Estudante:

Turma / Data:

🎯 **Objetivo:** Compreender como o Back-end protege dados e rotas.

Regra do Laboratório: Não vale copiar e colar tudo! Digite bloco por bloco no **VS Code**. Use a **"Cola Funcional"** abaixo para descobrir o que preencher em cada lacuna amarela.

1. 🟢 A COLA FUNCIONAL (SEU BANCO DE COMANDOS)

Aqui estão os pedaços de código que estão faltando no servidor. Estude o que cada um faz e encaixe-os no local correto:

bcrypt.hash(password, 10)

• **O que faz:** Pega a senha em texto limpo e transforma em um código embaralhado (Hash) irreversível.

bcrypt.compare(password, usuario.passwordHash)

• **O que faz:** Testa se a senha digitada no login gera o mesmo Hash que foi guardado no banco.

jwt.sign({ id: usuario.id, role: usuario.role }, CHAVE)

• **O que faz:** Cria e assina o "crachá digital" (Token JWT) contendo quem é o usuário e qual o cargo dele.

req.headers['authorization']

• **O que faz:** Captura o Token que o aplicativo/cliente enviou pelo cabeçalho secreto da requisição.

jwt.verify(token, CHAVE, (err, decodificado) => { ... })

• **O que faz:** Inspecciona o Token. Se alguém alterou uma letra dele ou se estiver expirado, ele bloqueia o acesso.

decoded.role

• **O que faz:** Lê qual era o cargo/perfil gravado de forma segura dentro do Token já validado.

2. 🖥️ O CÓDIGO DO SERVIDOR (CRIE O ARQUIVO APP.JS NO VS CODE)

Instale os pacotes no terminal: `npm init -y` e depois `npm install express bcrypt jsonwebtoken`. Digite o código abaixo preenchendo as lacunas:

```
const express = require('express');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

const app = express();
app.use(express.json());

const CHAVE = "CPS_SECRET_2026";
const usuarios = []; // Banco de dados simulado

// --- PASSO 1: CADASTRO DE USUÁRIO (Criptografar para proteger) ---
app.post('/cadastro', async (req, res) => {
  const { username, password, role } = req.body;

  // [LACUNA A] Transforma a senha digitada em um Hash seguro antes de salvar
  const senhaProtegida = await /* ??? A */;

  usuarios.push({ username, passwordHash: senhaProtegida, role });
```

```
    res.status(201).send("Usuário cadastrado com sucesso!");
  });

// --- PASSO 2: LOGIN (Validar a senha e entregar o Crachá/Token) ---
app.post('/login', async (req, res) => {
  const { username, password } = req.body;
  const usuario = usuarios.find(u => u.username === username);
  if (!usuario) return res.status(400).send("Usuário não encontrado.");

  // [LACUNA B] Compara a senha digitada com o Hash guardado no banco
  const senhaCorreta = await /* ??? B */;
  if (!senhaCorreta) return res.status(401).send("Senha incorreta.");

  // [LACUNA C] Cria o token JWT (Crachá Digital) com os dados e cargo do usuário
  const tokenGerado = /* ??? C */;
  res.json({ token: tokenGerado });
});
```

```
// --- PASSO 3: SEGURANÇA DA PORTA (Middleware que inspeciona o Crachá) ---
function verificarAcesso(rolesPermitidas) {
  return (req, res, next) => {
    // [LACUNA D] Captura o token enviado no cabeçalho de autorização da requisição
    const tokenEnviado = /* ??? D */;
    if (!tokenEnviado) return res.status(403).send("Acesso negado: Sem token.");

    // [LACUNA E] Valida se o token é legítimo e lê os dados decodificados
    /* ??? E */ (tokenEnviado, CHAVE, (err, decoded) => {
      if (err) return res.status(401).send("Token inválido ou adulterado.");

      // [LACUNA F] Extrai o cargo (role) de dentro do token decodificado
      const cargoUsuario = /* ??? F */;

      // Verifica se o cargo do usuário está na lista de permissões daquela rota
      if (!rolesPermitidas.includes(cargoUsuario)) {
        return res.status(403).send("Acesso proibido para o seu perfil.");
      }

      next(); // Tudo certo! Autoriza a passar para a rota.
    });
  };
}

// --- PASSO 4: DEFINIÇÃO DAS ROTAS PROTEGIDAS POR CARGO ---
app.get('/admin/painel', verificarAcesso(['Administrador']), (req, res) => {
  res.send("Área restrita da Diretoria acessada!");
});

app.get('/vendas', verificarAcesso(['Administrador', 'Vendedor']), (req, res) => {
  res.send("Painel de Vendas liberado.");
});

app.listen(3000, () => console.log("Servidor Defensivo rodando na porta 3000"));
```

3. DESAFIO DE ANÁLISE (RESPONDA NO CADERNO)

1. Olhando para o seu código e para a **Cola Funcional**, qual a diferença prática entre o comando que roda na **[LACUNA A]** (cadastro) e o comando que roda na **[LACUNA B]** (login)? Por que usamos comandos diferentes para a senha nesses dois momentos?
2. Imagine que um aluno malicioso descobriu a URL `http://localhost:3000/admin/painel` e tentou acessá-la diretamente pelo navegador sem fazer login. Explique qual linha do código vai barrar esse acesso e qual será a mensagem de erro que ele vai receber na tela.
3. Se você criar um usuário com o cargo (role) de "Vendedor", ele conseguirá visualizar o que está na rota `/admin/painel`? Justifique sua resposta analisando a linha onde a função `verificarAcesso` é configurada.